



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of computer science engineering and information systems

SCORE: Fall Semester 2026-2027

NAME: MENAKA.A

REG NO: 24MIS0431

FACULTY NAME: DR PRASANNA.M

COURSE CODE: ISWE305P

COURSE NAME: SOFTWARE TESTING LAB

LAB SLOT: L47+L48

1) A Nuclear Power Plant Reactor Monitoring System continuously supervises critical reactor parameters to ensure safe plant operation and prevent catastrophic failures. The system monitors the following variables:

- Reactor Core Temperature: 200°C to 1200°C
 - Coolant Level: 20% to 100%
 - Radiation Level: 0 mSv to 500 mSv
 - Reactor Pressure: 100 kPa to 1000 kPa
- The system automatically triggers alarms, activates emergency cooling mechanisms, or initiates reactor shutdown procedures whenever unsafe operating conditions are detected.

Tasks to be Performed:

1. Identify all boundary values associated with each monitored parameter.
2. Generate Boundary Value Analysis (BVA) test cases for the reactor monitoring application.
3. Generate Robust Testing test cases by considering invalid values below and above the permissible limits.
4. Identify combinations of parameter values that could result in reactor malfunction, radiation leakage, or complete mission failure, and specify the corresponding emergency actions that should be initiated by the system.

Algorithm

1. Start.
2. Read reactor temperature, coolant level, radiation level, and pressure.
3. Check whether all values are within their specified limits.
4. If all values are valid, display "**Reactor Status: SAFE**".
5. Otherwise, display "**ALERT!**", "**Emergency Cooling Activated**", and "**Reactor Shutdown Initiated**".
6. Stop.

Java program:

```
import java.util.Scanner;

public class NuclearReactor {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Temperature: ");
        int temp = sc.nextInt();
```

```
System.out.print("Coolant Level: ");  
int coolant = sc.nextInt();
```

```
System.out.print("Radiation Level: ");  
int radiation = sc.nextInt();
```

```
System.out.print("Pressure: ");  
int pressure = sc.nextInt();
```

```
if (temp>=200 && temp<=1200 &&  
    coolant>=20 && coolant<=100 &&  
    radiation>=0 && radiation<=500 &&  
    pressure>=100 && pressure<=1000) {
```

```
    System.out.println("Reactor Status: SAFE");  
} else {  
    System.out.println("ALERT!");  
    System.out.println("Emergency Cooling Activated");  
    System.out.println("Reactor Shutdown Initiated");  
}
```

```
    sc.close();  
}  
}
```

Output:

POSITIVE OUTPUT:

```
PS C:\Users\menak> javac NuclearReactor.java
PS C:\Users\menak> java NuclearReactor
Temperature: 200
Coolant Level: 20
Radiation Level: 0
Pressure: 100
Reactor Status: SAFE
```

NEGATIVE OUTPUT:

```
Reactor Status: SAFE
PS C:\Users\menak> javac NuclearReactor.java
PS C:\Users\menak> java NuclearReactor
Temperature: 199
Coolant Level: 50
Radiation Level: 100
Pressure: 500
ALERT!
Emergency Cooling Activated
Reactor Shutdown Initiated
```

DESIGN A TESTCASE MATRIX TABLE

Test Case	Temperature (200–1200°C)	Coolant (20–100%)	Radiation (0–500 mSv)	Pressure (100–1000 kPa)	Expected Result	Actual Result	Status
TC01	200	20	0	100	Safe Operation	Safe Operation	pass
TC02	1200	100	500	1000	Safe Operation	Safe Operation	pass
TC03	199	50	100	500	Alarm + Emergency Cooling +	Alarm + Emergency Cooling +	pass

					Shutdown	Shutdown	
TC O4	1201	50	100	500	Alarm + Emergency Cooling + Shutdown	Alarm + Emergency Cooling + Shutdown	pass
TC O5	800	19	100	500	Alarm + Emergency Cooling + Shutdown	Alarm + Emergency Cooling + Shutdown	pass
TC O6	800	101	100	500	Alarm + Radiation Leakage Warning + Shutdown	Alarm + Radiation Leakage Warning + Shutdown	pass
TC O7	800	50	100	1001	Alarm + Reactor Malfunction + Shutdown	Alarm + Reactor Malfunction + Shutdown	pass

2. A Smart Elevator Monitoring System continuously monitors the operating conditions of an elevator in a commercial building. The system checks the following parameters:

- Number of Passengers: 0 to 15 persons
- Load Weight: 0 kg to 1000 kg
- Elevator Speed: 0 m/s to 5 m/s
- Cabin Temperature: 15°C to 40°C The system generates warning messages whenever any parameter exceeds its permissible limit and may stop the elevator to ensure passenger safety.

Tasks to be Performed:

1. Identify all boundary values for each input parameter.
2. Generate Boundary Value Analysis (BVA) test cases.
3. Generate Robust Testing test cases by considering values just below and above the valid ranges.
4. Determine which combinations of parameter values may lead to system failure or unsafe conditions, such as:
 - o Number of passengers exceeding 15.
 - o Load weight greater than 1000 kg.
 - o Excessive elevator speed combined with overload conditions.
 - o Extremely high cabin temperature and overload occurring simultaneously.

Implement the above application and analyze it using Boundary Value Analysis (BVA) and Robust Testing techniques.

Algorithm

1. Start.
2. Read the number of passengers, load weight, elevator speed, and cabin temperature.

3. Check whether all values are within the allowed range.
4. If all values are valid, display "**Elevator Status: SAFE**".
5. Otherwise, display "**WARNING!**" and "**Elevator Stopped**".
6. Stop.

Java program:

```
import java.util.Scanner;

public class SmartElevator {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Passengers: ");

        int passengers = sc.nextInt();

        System.out.print("Load Weight: ");

        int weight = sc.nextInt();

        System.out.print("Speed: ");

        double speed = sc.nextDouble();

        System.out.print("Cabin Temperature: ");

        int temp = sc.nextInt();

        if(passengers>=0 && passengers<=15 &&
            weight>=0 && weight<=1000 &&
            speed>=0 && speed<=5 &&
            temp>=15 && temp<=40){
```

```

        System.out.println("Elevator Status: SAFE");
    }

    else{

        System.out.println("WARNING!");

        System.out.println("Elevator Stopped");

    }

    sc.close();

}

}

```

Output:

POSITIVE OUTPUT:

```

PS C:\Users\menak> javac SmartElevator.java
PS C:\Users\menak> java SmartElevator
Passengers: 0
Load Weight: 0
Speed: 0
Cabin Temperature: 15
Elevator Status: SAFE

```

NEGATIVE OUTPUT:

```

PS C:\Users\menak> javac SmartElevator.java
PS C:\Users\menak> java SmartElevator
Passengers: -1
Load Weight: 500
Speed: 2
Cabin Temperature: 25
WARNING!
Elevator Stopped

```

Test Case	Passengers (0–15)	Load Weight (0–	Speed (0–5 m/s)	Cabin Temp (15–	Expected Result	Actual result	status

		1000 kg		40°C)			
TCO 1	0	0	0	15	Elevator Safe (Minimum Valid)	Elevator Safe (Minimum Valid)	pass
TCO 2	15	1000	5	40	Elevator Safe (Maximum Valid)	Elevator Safe (Maximum Valid)	pass
TCO 3	-1	500	2	25	Warning – Elevator Stopped (Invalid Passengers)	Warning – Elevator Stopped (Invalid Passengers)	pass
TCO 4	16	500	2	25	Warning – Elevator Stopped (Passenger Limit Exceeded)	Warning – Elevator Stopped (Passenger Limit Exceeded)	pass
TCO 5	10	-1	2	25	Warning – Elevator Stopped (Invalid Load Weight)	Warning – Elevator Stopped (Invalid Load Weight)	pass
TCO 6	10	1001	2	25	Warning – Elevator Stopped (Overload ed)	Warning – Elevator Stopped (Overload ed)	pass
TCO 7	10	800	5.5	25	Warning – Elevator	Warning – Elevator	pass

					Stopped (Overspeed)	Stopped (Overspeed)	
TCO 8	10	800	3	41	Warning – Elevator Stopped (High Cabin Temperature)	Warning – Elevator Stopped (High Cabin Temperature)	pass

3. An Online Hotel Reservation Management System is developed to automate the booking process for customers reserving hotel rooms through a web-based platform. The system evaluates each booking request based on several business conditions, including room availability, payment status, customer membership category, booking amount, and availability of complimentary services. For every reservation request, the system checks whether the requested room is available, whether the payment transaction has been completed successfully, whether the customer is a loyalty member, whether the total booking amount exceeds ₹20,000, and whether airport pickup service is available. Based on the evaluation of these conditions, the system performs one or more of the following actions:

1. Confirm Reservation – The reservation is confirmed only if the room is available and payment has been successfully completed.
2. Provide Complimentary Benefits – Complimentary services such as free breakfast or room upgrades are provided if the customer is a loyalty member or if the booking amount exceeds ₹20,000.
3. Enable Airport Pickup Service – Airport pickup is enabled if the service is available and the reservation has been confirmed.

4. Put Reservation on Hold – The reservation is temporarily placed on hold if the room is available but payment verification is pending, allowing the customer to complete the payment within a specified time.

5. Cancel Reservation – The reservation is cancelled if:

- o The requested room is unavailable.
- o Payment is permanently declined.

- o The customer fails to complete payment within the permitted time.

Tasks to be Performed:

1. Identify all possible combinations of the input conditions and construct a comprehensive decision table.

2. Based on the decision table, generate all relevant test cases.

3. Implement the application and verify that all actions conform to the defined business rules.

Algorithm

1. Start.

2. Read room availability, payment status, loyalty membership, booking amount, and airport pickup availability.

3. If the room is available and payment is successful:

- o Confirm the reservation.
- o If the customer is a loyalty member or the booking amount is greater than ₹20,000, provide complimentary benefits.
- o If airport pickup is available, enable airport pickup.

4. Else if the room is available but payment is unsuccessful, place the reservation on hold.

5. Otherwise, cancel the reservation.

6. Stop.

Java program:

```
import java.util.Scanner;

public class HotelReservation {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Room Available (true/false): ");

        boolean room = sc.nextBoolean();

        System.out.print("Payment Successful (true/false): ");

        boolean payment = sc.nextBoolean();

        System.out.print("Loyalty Member (true/false): ");

        boolean member = sc.nextBoolean();

        System.out.print("Booking Amount: ");

        double amount = sc.nextDouble();

        System.out.print("Airport Pickup Available (true/false): ");

        boolean pickup = sc.nextBoolean();

        if(room && payment){

            System.out.println("Reservation Confirmed");

            if(member || amount>20000)

                System.out.println("Complimentary Benefits Provided");

            if(pickup)

                System.out.println("Airport Pickup Enabled");
```

```

    }

    else if(room && !payment){

        System.out.println("Reservation On Hold");

    }

    else{

        System.out.println("Reservation Cancelled");

    }

    sc.close();

}

}

```

Output:

POSITIVE OUTPUT:

```

PS C:\Users\menak> javac HotelReservation.java
PS C:\Users\menak> java HotelReservation
Room Available (true/false): true
Payment Successful (true/false): true
Loyalty Member (true/false): true
Booking Amount: 21000
Airport Pickup Available (true/false): true
Reservation Confirmed
Complimentary Benefits Provided
Airport Pickup Enabled

```

NEGATIVE OUTPUT:

```

PS C:\Users\menak> javac HotelReservation.java
PS C:\Users\menak> java HotelReservation
Room Available (true/false): true
Payment Successful (true/false): false
Loyalty Member (true/false): true
Booking Amount: 21000
Airport Pickup Available (true/false): true
Reservation On Hold

```

Test	Room	Payment	Loyalty	Booking	Airport	Expected Result	Actual result	status
------	------	---------	---------	---------	---------	-----------------	---------------	--------

case	Available	Successful	Member	Amount (>₹20,000)	Pickup Available			
TC O1	true	true	true	21000	true	Reservation Confirmed + Complimentary Benefits + Airport Pickup	Reservation Confirmed + Complimentary Benefits + Airport Pickup	passes
TC O2	true	true	false	-	true	Reservation Confirmed + Airport Pickup	Reservation Confirmed + Airport Pickup	passes
TC O3	true	true	true	22000	false	Reservation Confirmed + Complimentary Benefits	Reservation Confirmed + Complimentary Benefits	passes
TC O4	true	false	true	23000	true	Reservation Confirmed + Complimentary Benefits	Reservation Confirmed + Complimentary Benefits	passes
TC O5	false	true	true	24000	true	Reservation	Reservation	passes

						Canceled	Canceled	
TC O6	true	true	true	19000	false	Reservation Confirmed + Complimentary Benefits	Reservation Confirmed + Complimentary Benefits	pass
TC O7	true	true	false	18000	false	Reservation Confirmed	Reservation Confirmed	pass

4. An Online University Course Registration System is developed to automate the process of student course enrollment through a web-based platform. The system evaluates each registration request based on several academic and administrative conditions, including seat availability, fee payment status, student eligibility, course credit limit, and availability of laboratory sessions. For every registration request, the system checks whether the requested course has available seats, whether the semester fee has been paid successfully, whether the student satisfies the prerequisite requirements, whether the total registered credits exceed 20 credits, and whether the laboratory session is available for the selected course. Based on the evaluation of these conditions, the system performs one or more of the following actions:

1. Confirm Registration – The course registration is confirmed only when seats are available, prerequisites are satisfied, and the semester fee has been paid successfully.

2. Provide Priority Enrollment – Priority enrollment benefits are provided if the student is academically eligible and registers for more than 20 credits or belongs to the honors program.

3. Enable Laboratory Registration – Laboratory registration is enabled when laboratory sessions are available and the course registration has been successfully confirmed.

4. Put Registration on Hold – The registration request is temporarily placed on hold if seats are available but fee verification is pending, allowing the student to complete the payment within the specified period.

5. Cancel Registration – The registration is cancelled under the following circumstances:

- o No seats are available in the selected course.
- o The student does not satisfy the prerequisite requirements.
- o Fee payment fails permanently or is not completed within the permitted time.

Tasks to be Performed:

1. Identify all possible combinations of the input conditions and construct a comprehensive decision table for the Online University Course Registration System.

2. Based on the decision table, generate all relevant test cases to verify the correctness and reliability of the registration application.

3. Implement the application and validate whether the actions performed by the system conform to the defined business rules.

Algorithm

1. Start.
2. Read prerequisite completion status, seat availability, fee payment status, and attendance eligibility.
3. Check whether all conditions are satisfied.
4. If all conditions are true, display **"Registration Successful"**.
5. Otherwise, display **"Registration Failed"**.
6. Stop.

Java program:

```
import java.util.Scanner;

public class CourseRegistration {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Prerequisite Completed (true/false): ");

        boolean prerequisite = sc.nextBoolean();

        System.out.print("Seats Available (true/false): ");

        boolean seats = sc.nextBoolean();

        System.out.print("Fees Paid (true/false): ");

        boolean fees = sc.nextBoolean();

        System.out.print("Attendance Eligible (true/false): ");

        boolean attendance = sc.nextBoolean();

        if(prerequisite && seats && fees && attendance){

            System.out.println("Registration Successful");
```

```

    }

    else{

        System.out.println("Registration Failed");

    }

    sc.close();

}

}

```

OUTPUT:

POSITIVE OUTPUT:

```

PS C:\Users\menak> javac CourseRegistration.java
PS C:\Users\menak> java CourseRegistration
Prerequisite Completed (true/false): true
Seats Available (true/false): true
Fees Paid (true/false): true
Attendance Eligible (true/false): true
Registration Successful

```

NEGATIVE OUTPUT:

```

PS C:\Users\menak> javac CourseRegistration.java
PS C:\Users\menak> java CourseRegistration
Prerequisite Completed (true/false): true
Seats Available (true/false): false
Fees Paid (true/false): true
Attendance Eligible (true/false): true
Registration Failed

```

Test case	Prerequisite completed	Seats available	Fees paid	Attendance eligible	Expected output	Actual output	status
TC 1	yes	yes	yes	yes	Registration	Registration	pass

					Successful	Successful	
TC 2	no	yes	yes	yes	Registration Failed (Prerequisite Not Completed)	Registration Failed (Prerequisite Not Completed)	pass
TC 3	yes	no	yes	yes	Registration Failed (No Seats Available)	Registration Failed (No Seats Available)	pass
TC 4	yes	yes	no	yes	Registration Failed (Fees Not Paid)	Registration Failed (Fees Not Paid)	pass
TC 5	yes	yes	yes	no	Registration Failed (Attendance Not Eligible)	Registration Failed (Attendance Not Eligible)	pass
TC 6	no	no	yes	yes	Registration Failed	Registration Failed	pass
TC 7	yes	no	no	yes	Registration Failed	Registration Failed	pass